Context & Objective

You are an expert AI assistant specializing in Medical Image Analysis, specifically using the nnUNetv2 framework.

My Objective: I need to build a baseline 3D segmentation model for detecting clinically significant prostate cancer (csPCa) using the PI-CAI dataset. I will be training an nnU-Net model.

I need you to help me write the Python scripts to format my data, set up my environment, and generate the exact terminal commands to run the nnU-Net pipeline.

1. Data Source (Current State)

My data is currently stored in a Google Drive folder named PI-CAI_pre-processed. The files are in NIfTI format (.nii.gz). The directory structure is as follows:

- t2/ (T2-weighted images)
- adc_reg/ (Apparent Diffusion Coefficient maps, co-registered to T2)
- hbv_reg/ (High b-value DWI, co-registered to T2)
- lesion_masks/ (Ground truth annotations for csPCa)
- zonal_masks/ & whole_gland_masks/ (Anatomical priors - currently ignoring for this baseline)
- clinical_information/ & metadata/ (Contains patient IDs and study IDs)

Filename convention: [patientID]_[studyID].nii.gz (e.g., 10001_1000001.nii.gz)

2. Target Data Structure (nnU-Net v2 Format)

You must help me write a Python script to copy and rename the files from my Drive structure into the strict nnUNet_raw format. We will designate this as **Dataset500_PICAI**.

The output structure must look exactly like this:

```
nnUNet_raw/
├── Dataset500_PICAI/
│   ├── dataset.json
│   ├── imagesTr/
│   │   ├── 10001_1000001_0000.nii.gz  (Mapped from t2/)
│   │   ├── 10001_1000001_0001.nii.gz  (Mapped from adc_reg/)
│   │   ├── 10001_1000001_0002.nii.gz  (Mapped from hbv_reg/)
│   │   ├── ...
│   ├── labelsTr/
│   │   ├── 10001_1000001.nii.gz      (Mapped from lesion_masks/)
│   │   ├── ...
```

3. Critical Constraints & Pitfalls (MUST READ)

When you write the Python data conversion script for me, you **must** implement logic to handle the following dataset-specific traps:

A. The "Missing Mask" Trap (Negative Cases)

- **Problem:** There are 1500 T2/ADC/HBV scans, but only 1293 lesion masks. The 207 missing masks belong to healthy/benign patients. If we skip them, the model will suffer from extreme positive bias.
- **Solution requirement for the script:** When iterating through the t2/ folder, check if the corresponding mask exists in lesion_masks/. If it does **NOT** exist, the script must load the T2 NIfTI file (using nibabel or SimpleITK), create an empty numpy array (all zeros) of the exact same shape and affine, and save it to the labelsTr/ folder.

B. The "Data Leakage" Trap (Longitudinal Scans)

- **Problem:** There are 24 patients with multiple studies (e.g., scan in 2021 and 2022). If we let nnU-Net do a random 5-fold split, a patient's 2021 scan might end up in the training set and their 2022 scan in the validation set, causing data leakage.
- **Solution requirement for the script:** You must write a script to generate a custom splits_final.json for nnUNetv2. The script must group the dataset by patientID (the first part of the filename) and perform an 80/20 GroupKFold split (5 folds). All studyIDs belonging to the same patientID must remain in the same fold.

C. Interpolation Trap

- **Problem:** If any spatial resampling is done to masks before feeding them to nnU-Net, decimal labels are created.
- **Constraint:** Do not apply any resizing in our Python script. Simply move/rename the images and generate the blank masks. We will rely on nnU-Net's internal nnUNetv2_plan_and_preprocess command to handle spatial standardization.

4. Execution Tasks for the AI

Based on the context above, please provide the following:

1. **Environment Setup:** The exact bash commands to set up the three required nnUNet environment variables (nnUNet_raw, nnUNet_preprocessed, nnUNet_results).
2. **Data Conversion Script:** A complete, robust Python script that reads from my Google Drive paths, formats the Dataset500_PICAI folder, renames the 3 modalities with _0000, _0001, _0002 suffixes, and generates the blank NIfTI masks for the negative cases.
3. **JSON Generation Script:** A Python script to generate the required dataset.json (defining channel names, labels, and file counts) AND the custom splits_final.json (enforcing

patient-level grouping for 5 folds).

4. **Training Commands:** The exact terminal commands I need to run to extract the dataset fingerprints, run the automated preprocessing, and trigger the 3D Full-Resolution training for Fold 0.